1. Introduction

This document has been prepared so that contestants can run, in their own environments, the environment we prepared to test the minimum functionality of designs that have reached the DDR phase. The following sections explain the minimum functionality expected from the design, the contents of the test environment, and how to prepare and run it.

2. Minimum Design Requirements

To run the test, the following modules in your design are expected to be operational.

- CV32E40P Core
- UART0 (the AI accelerator UART is not required)
- Boot ROM (QSPI is not required; if available, you may use it, but since we configured the testbench to boot using only the boot ROM, bypassing the QSPI section will make your work easier.)
- DATA SRAM (required for the test code to run)

3. Test Environment Contents

The test environment to be provided to the contestants is as follows. Copy the folder into your repository and update the files under the `user_files` folder as described in Section 4.

```
teknotest
│
├──scripts
│   └──create_vivado_proj.tcl // Creates the Vivado project for the
│                             // simulation environment
├──sw
│   ├──scripts
│   │   ├──build.py      // Compiles the C code
│   │   └──elf_to_mem.py // Converts the generated .elf file to a Vivado
│   │                    // simulation-compatible .mem file
│   └──src
│       ├──crt0.S        // Core entry code
│       └──helloworld.c  // Test application
│
├──tb
│   └──teknotest_tb.sv   // Testbench
│
└──user_files // Modify only the files here. Explanation below
    ├──bootrom.ld
    ├──compile_user_design.tcl
    ├──rv_toolchain.conf
    ├──teknotest_tb_user_code.sv
    ├──teknotest_wrapper.sv
    └──user_defines.h
```

IMPORTANT NOTE: Only update the files under the `user_files` folder. Making changes in other folders may cause your test environment to fail during the evaluation by the judges. If you truly believe that you need to make changes in other files, you may consult DDK.

4. Preparing the Test Environment

To run the tests correctly, you must configure the files under the `user_files` folder according to your own design and environment.

4.1. Preparing the Software Environment

To set up the software environment, the following 3 files must be updated:

- `bootrom.ld`
- `rv_toolchain.conf`
- `user_defines.h`

First, in the `bootrom.ld` linker file, replace the `#BOOTROM_BASE_ADDR#` and `#DATA_RAM_BASE_ADDR#` fields in the area shown below with your Boot ROM (or instruction SRAM) and data SRAM addresses, respectively.

```
11  MEMORY
12  {
13    BOOTROM (rx)  : ORIGIN = #BOOTROM_BASE_ADDR#, LENGTH = 1K
14    RAM     (rwx) : ORIGIN = #DATA_RAM_BASE_ADDR#, LENGTH = 8K
15  }
```

For example:

```
11  MEMORY
12  {
13    BOOTROM (rx)  : ORIGIN = 0x00000000, LENGTH = 1K
14    RAM     (rwx) : ORIGIN = 0x80000000, LENGTH = 8K
15  }
```

NOTE: Do not make changes to any area of the linker file other than this area.

Update the `rv_toolchain.conf` file to use the toolchain in your environment. The `build.py` script reads this file and uses the toolchain with the specified prefix at the specified file path.

You can uncomment the examples given below according to your operating system and point them to your own toolchain path in a similar format.

```
1  # Define your RISC-V toolchain path here
2  # Linux example:
3  # RISCV_GCC_PREFIX = "/opt/riscv/bin/riscv32-unknown-elf"
4  #
5  # Windows example:
6  # RISCV_GCC_PREFIX = r"C:\RISC-V\bin\riscv32-unknown-elf"
```

NOTE: Unlike the other files, this file will be updated again during the evaluation by the judges. You do not need to provide a valid path for the judges in this file; it is sufficient for it to work in your environment.

NOTE: The path you assign to the `RISCV_GCC_PREFIX` variable is not a complete folder path. This variable works as a prefix. For example, suppose the full path of your gcc compiler on Linux is `/home/my_pc/riscv_toolchain/bin/riscv32-corev-elf-gcc`. In this case, you must specify the toolchain path as `RISCV_GCC_PREFIX = "/home/my_pc/riscv_toolchain/bin/riscv32-corev-elf"`.

In addition, the Python script automatically adds the .exe suffixes for Windows. Therefore, you do not need an extra specifier for them.

Finally, in the `user_defines.h` file, you must assign the base address of the UART in your own architecture to the `UART_BASE_ADDR` constant. This header file is included in `helloworld.c`, and UART accesses are performed accordingly.

4.2. Preparing the Testbench Environment

To set up the testbench environment, the following 3 files must be updated:

- `teknotest_wrapper.sv`
- `teknotest_tb_user_code.sv`
- `compile_user_design.tcl`

First, instantiate your design inside `teknotest_wrapper.sv` so that it can be used in the testbench environment. The 4 pins of the wrapper module are described in the table below. If your design contains pins other than these 4 pins, they must be tied to 0/1 inside the wrapper module or, if they need to be driven dynamically, they can be driven inside `teknotest_tb_user_code.sv`.

Pin Name	Direction	Description
clk_i	Input	Clock signal input
resetn_i	Input	Reset signal input (Active Low)
uart_rx_i	Input	UART RX Input (testbench->wrapper)
uart_tx_o	Output	UART TX Output (wrapper->testbench)

The code you write inside the `teknotest_tb_user_code.sv` file is included inside `teknotest_tb.sv`. In this file, you can load the test code into the Boot ROM (or instruction SRAM), and if there are signals that need to be driven for the system to boot, the required code can be written here. An example of how to use the `teknotest_tb_user_code.sv` file is given below.

```
// Add your memory initialization code here.
// Example:
//  initial $readmemh("helloworld.mem", dut.i_bootrom.rom);
//
// You MUST KEEP "helloworld.mem" file constant, since it will be automatically
// added to the Vivado project, you should only change boot ROM memory array path.
// Also you can add processes needed for your system to boot
```

NOTE: Because the `helloworld.mem` file is added to the Vivado project inside the `create_vivado_proj.tcl` script, the name of this file must remain constant. Apart from this, writing the memory initialization code is left to the requirements of the contestants.

Finally, to compile your design, you must edit the `compile_user_design.tcl` file. In this file, you can add your design files to the project, define include files, and define any macros that you need to use in the code. An example of how to use the `compile_user_design.tcl` file is given below.

```
# Add your design files into the Vivado project
#
# To add design files, use
#   add_files <PATH_TO_YOUR_DESIGN_FOLDER>
# Example:
#   add_files ../user/rtl/teknotest_wrapper.sv
#
# To add include directories, use
#   set_property include_dirs [file normalize <PATH_TO_YOUR_INCLUDE_FOLDER>] [list [get_filesets sources_1] [get_filesets sim_1]]
# Example:
#   set_property include_dirs [file normalize "../user/rtl/core/include/"] [list [get_filesets sources_1] [get_filesets sim_1]]
#
# If you need to add verilog defines, use
#   set_property verilog_define {<YOUR_DEFINES>} [list [get_filesets sources_1] [get_filesets sim_1]]
# Example:
#   set_property verilog_define {SIMULATION USE_CG_BEHAV_MODELS} [list [get_filesets sources_1] [get_filesets sim_1]]
#
# Note1: Keep in mind that you will be in teknotest directory, set your file/folder paths relative to that directory
# Note2: You can use slashes(/) in Windows too to seperate the paths
```

IMPORTANT NOTE: When specifying a file path, you must provide a relative path with respect to the `teknotest` folder instead of an absolute path. Scripts containing absolute paths will not work during the evaluation by the judges, which will cause you to be considered as having failed the test.

5. Running the Test

The test scenario to be run will test the UART RX and TX lines. The test flow proceeds as follows.

0. The testbench provides 50 MHz clock and reset signals to the wrapper module, bringing up the module to be tested. (Since the C code calculates the baud rate based on a 50 MHz clock, if your design includes a PLL, remember to configure it to generate a 50 MHz clock as well.)
1. The CPU brings up the UART controller and starts sending the character ‘R’ on the TX line.
2. On the testbench side, the character ‘R’ is waited for, and after it is received, the character ‘A’ is sent.
3. The CPU waits for the character ‘A’; after the character is received, it starts sending the “Hello World!” message.
4. The testbench waits for the “Hello World!” message. At this point, 3 different scenarios can occur.
    a. The “Hello World!” message is received as expected, and the test completes successfully.
    b. If a different message with the same length as the “Hello World!” message is received, the expected message and the received message are printed to the screen, and the test fails.
    c. If the “Hello World!” message, or a message with the same length, is not received within the 10 ms period defined in the testbench, the test times out and fails.

To run the test, the test software must first be compiled. Open a terminal and run the `scripts/build.py` script under the `sw` folder to compile the test software. An example command is given below.

```
PS C:\Users\bcakar\Documents\teknotest_env\teknotest\sw> & C:/Users/bcakar/AppData/Local/Programs/Python/Python39/python.exe ./scripts/build.py
```

If everything goes correctly, a message like the following will appear, indicating that the build completed successfully.

```
Build completed successfully.
ELF     : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.elf
MAP     : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.map
DISASM  : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.dis
READELF : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.readelf
MEM     : C:\Users\bcakar\Documents\teknotest_env\teknotest\sw\build\helloworld.mem
```

After compiling the test software, open Vivado and run the following commands in the TCL console. Before running the first command, replace the `<teknotest_folder_path>` variable with the path to the `teknotest` folder on your own system.

```
cd <teknotest_folder_path>
source ./scripts/create_vivado_proj.tcl
```

The script run by the second command will automatically create the Vivado project and open it. After the project opens, run the simulation until completion. A successful simulation should produce the following output.